sgs reproducible example

Fabio Feser

2024-04-26

Introduction

The `sgs` R package fits sparse-group SLOPE (SGS) and group SLOPE (gSLOPE) models. The package has implementations for linear and logisitic regression, both of which are demonstrated here. The package also uses strong screening rules to speed up computational time, described in detail in F. Feser, M. Evangelou (2024) "Strong Screening Rules for Group-based SLOPE Models". Screening rules are applied by default here. However, the impact of screening is demonstrated in the Screening section at the end.

Sparse-group SLOPE

Sparse-group SLOPE (SGS) is a penalised regression approach that performs bi-level selection with FDR control under orthogonal designs. SGS is described in detail in F. Feser, M. Evangelou (2023) "Sparse-group SLOPE: adaptive bi-level selection with FDR-control".

Linear regression

Data

For this example, a 400×500 input matrix is used with a simple grouping structure, sampled from a multivariate Gaussian distribution with no correlation.

```
library(sgsDev)
groups = c(rep(1:20, each = 3), rep(21:40, each = 4), rep(41:60,
    each = 5), rep(61:80, each = 6), rep(81:100, each = 7))

data = gen_toy_data(p = 500, n = 400, groups = groups, seed_id = 3)
```

Fitting an SGS model

We now fit an SGS model to the data using linear regression. The SGS model has many different hyperparameters which can be tuned/selected. Of particular importance is the λ parameter, which defines the level of sparsity in the model. First, we select this manually and then next use cross-validation to tune it. The other parameters we leave as their default values, although they can easily be changed.

```
model = fit_sgs(X = data$X, y = data$y, groups = groups, type = "linear",
    lambda = 0.5, alpha = 0.95, vFDR = 0.1, gFDR = 0.1, standardise = "l2",
    intercept = TRUE, verbose = FALSE, screen = TRUE)
```

Note: we have fit an intercept and applied ℓ_2 standardisation. This is the recommended usage when applying SGS with linear regression. The lambda values can also be calculated automatically, starting at the null model and continuing as specified by `min_frac` and `path_length`:

```
model_path = fit_sgs(X = data$X, y = data$y, groups = groups,
    type = "linear", lambda = "path", path_length = 5, min_frac = 0.1,
```

```r
    alpha = 0.95, vFDR = 0.1, gFDR = 0.1, standardise = "l2",
    intercept = TRUE, verbose = FALSE, screen = TRUE)
```

Output of model

The package provides several useful outputs after fitting a model. The `beta` vector shows the fitted values (note the intercept). We can also recover the indices of the non-zero variables and groups, which are indexed from the first variable, not the intercept.

```r
model$beta[model$selected_var + 1]  # the +1 is to account for the intercept
```

```
##  [1] -1.6734720 -1.4893849  3.1678789  1.6350236  5.0369209 -1.4619740  1.2779414 -0.9364065  4.98093
## [11] -1.4470782 -2.2343330  1.6188817 -1.6665858 -0.7298015
```

```r
model$group_effects[model$selected_grp]
```

```
## [1] 3.879978 5.295647 1.461974 1.277941 5.068190 1.588364 2.234333 2.435343
```

```r
model$selected_var
```

```
##  [1]  97  99 100 133 136 170 217 231 234 260 263 334 391 393 394
```

```r
model$selected_grp
```

```
## [1] 30 39 46 56 59 64 76 85
```

Defining a function that lets us calculate various metrics (including the FDR and sensitivity):

```r
fdr_sensitivity = function(fitted_ids, true_ids, num_coef) {
    # calculates FDR, FPR, and sensitivity
    num_true = length(intersect(fitted_ids, true_ids))
    num_false = length(fitted_ids) - num_true
    num_missed = length(true_ids) - num_true
    num_true_negatives = num_coef - length(true_ids)
    out = c()
    out$fdr = num_false/(num_true + num_false)
    if (is.nan(out$fdr)) {
        out$fdr = 0
    }
    out$sensitivity = num_true/length(true_ids)
    if (length(true_ids) == 0) {
        out$sensitivity = 1
    }
    out$fpr = num_false/num_true_negatives
    out$f1 = (2 * num_true)/(2 * num_true + num_false + num_missed)
    if (is.nan(out$f1)) {
        out$f1 = 1
    }
    return(out)
}
```

Calculating relevant metrics give

```r
fdr_sensitivity(fitted_ids = model$selected_var, true_ids = data$true_var_id,
    num_coef = 500)
```

```
## $fdr
## [1] 0
##
```

```
## $sensitivity
## [1] 0.5357143
##
## $fpr
## [1] 0
##
## $f1
## [1] 0.6976744
```

```
fdr_sensitivity(fitted_ids = model$selected_grp, true_ids = data$true_grp_id,
    num_coef = 100)
```

```
## $fdr
## [1] 0
##
## $sensitivity
## [1] 0.8
##
## $fpr
## [1] 0
##
## $f1
## [1] 0.8888889
```

The model is currently too sparse, as our choice of λ is too high. We can instead use cross-validation.

Cross validation

Cross-validation is used to fit SGS models along a λ path of length 20. The first value, $\lambda_{\max}$, is chosen to give the null model and the path is terminated at $\lambda_{\min} = \delta\lambda_{\max}$, where δ is some value between 0 and 1 (given by `min_frac` in the function). The 1se rule (as in the `glmnet` package) is used to choose the optimal model.

```
cv_model = fit_sgs_cv(X = data$X, y = data$y, groups = groups,
    type = "linear", path_length = 20, nfolds = 10, alpha = 0.95,
    vFDR = 0.1, gFDR = 0.1, min_frac = 0.05, standardise = "l2",
    intercept = TRUE, verbose = TRUE, screen = TRUE)
```

```
## [1] "Fold 1/10 done"
## [1] "Fold 2/10 done"
## [1] "Fold 3/10 done"
## [1] "Fold 4/10 done"
## [1] "Fold 5/10 done"
## [1] "Fold 6/10 done"
## [1] "Fold 7/10 done"
## [1] "Fold 8/10 done"
## [1] "Fold 9/10 done"
## [1] "Fold 10/10 done"
```

The fitting verbose contains useful information, showing the error for each fold. Aside from the fitting verbose, we can see a more succinct summary by using the `print` function

```
print(cv_model)
```

```
##
##  regression type:
##
##          lambda     error estimated_non_zero
```

```
##  [1,] 1.8885960 9523.7357                 0
##  [2,] 1.6131093 9207.3047                 1
##  [3,] 1.3778075 8483.2428                 2
##  [4,] 1.1768288 7660.2673                 3
##  [5,] 1.0051665 6853.3743                 4
##  [6,] 0.8585444 5743.6287                14
##  [7,] 0.7333098 4449.9434                15
##  [8,] 0.6263430 3427.6404                15
##  [9,] 0.5349793 2666.8889                15
## [10,] 0.4569427 2096.7952                16
## [11,] 0.3902891 1624.7847                22
## [12,] 0.3333582 1237.4624                22
## [13,] 0.2847318  935.0866                22
## [14,] 0.2431984  714.0932                22
## [15,] 0.2077234  549.4968                24
## [16,] 0.1774231  423.4776                24
## [17,] 0.1515426  328.7270                26
## [18,] 0.1294373  254.9699                26
## [19,] 0.1105565  200.4616                27
## [20,] 0.0944298  160.1339                27
```

The best model is found to be the one at the end of the path:

```r
cv_model$best_lambda_id
```

```
## [1] 20
```

Checking the metrics again, we see how CV has generated a model with the correct amount of sparsity that gives FDR levels below the specified values.

```r
fdr_sensitivity(fitted_ids = cv_model$fit$selected_var, true_ids = data$true_var_id,
    num_coef = 500)
```

```
## $fdr
## [1] 0.03703704
##
## $sensitivity
## [1] 0.9285714
##
## $fpr
## [1] 0.002118644
##
## $f1
## [1] 0.9454545
```

```r
fdr_sensitivity(fitted_ids = cv_model$fit$selected_grp, true_ids = data$true_grp_id,
    num_coef = 100)
```

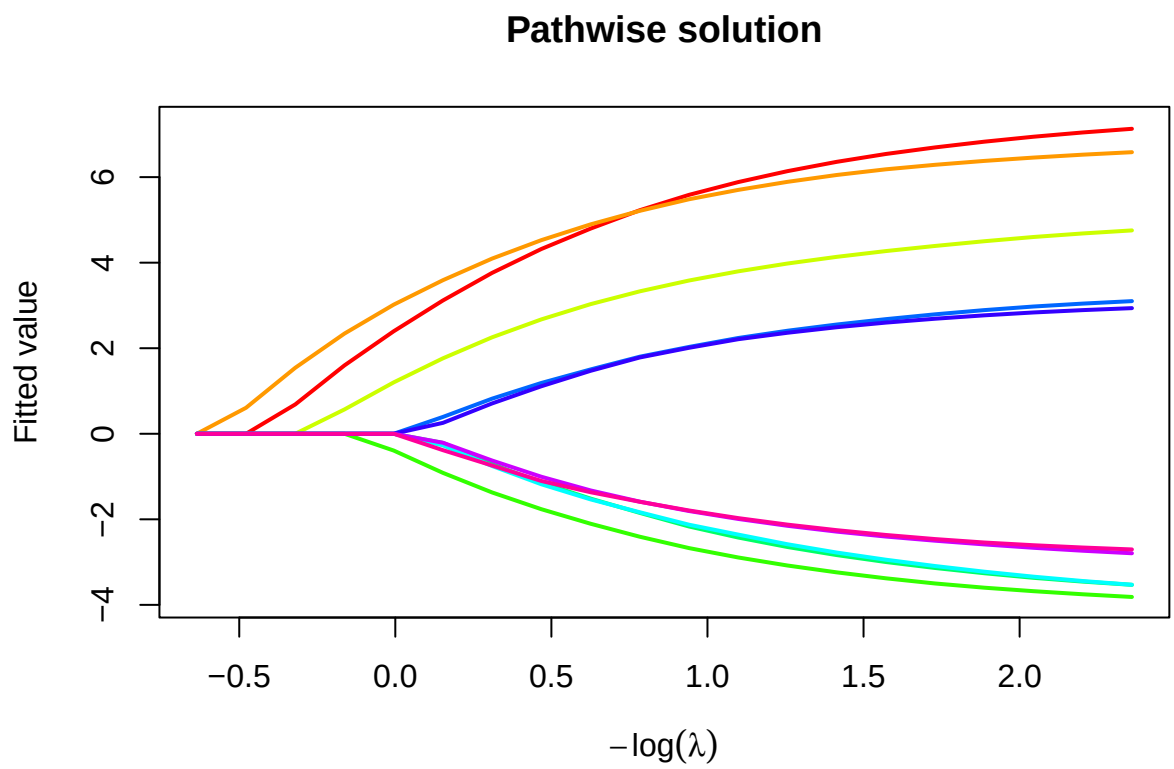
```
## $fdr
## [1] 0.09090909
##
## $sensitivity
## [1] 1
##
## $fpr
## [1] 0.01111111
##
```

```
## $f1
## [1] 0.952381
```

Plot

We can visualise the solution using the plot function:

```r
plot(cv_model, how_many = 10)
```



Prediction

The package has an implemented predict function to allow for easy prediction. The predict function can be used on regular and CV model fits.

```r
predict(model, data$X)[1:5]
```

```
## [1]  7.5631945 -2.5428616  7.8497662  0.8790488 -4.8362971
```

```r
dim(predict(cv_model, data$X))
```

```
## [1] 400  20
```

Logistic regression

As mentioned, the package can also be used to fit SGS to a binary response. First, we generate some binary data. We can use the same input matrix, X, and true β as before. We split the data into train and test to test the models classification performance.

```r
sigmoid = function(x) {
    1/(1 + exp(-x))
}
y = ifelse(sigmoid(data$X %*% data$true_beta + rnorm(400)) >
    0.5, 1, 0)
train_y = y[1:350]
test_y = y[351:400]
train_X = data$X[1:350, ]
test_X = data$X[351:400, ]
```

Fitting and prediction

We can again apply CV.

```r
cv_model = fit_sgs_cv(X = train_X, y = train_y, groups = groups,
    type = "logistic", path_length = 20, nfolds = 10, alpha = 0.95,
    vFDR = 0.1, gFDR = 0.1, min_frac = 0.05, standardise = "l2",
    intercept = FALSE, verbose = TRUE, screen = TRUE)
```

```
## [1] "Fold 1/10 done"
## [1] "Fold 2/10 done"
## [1] "Fold 3/10 done"
## [1] "Fold 4/10 done"
## [1] "Fold 5/10 done"
## [1] "Fold 6/10 done"
## [1] "Fold 7/10 done"
## [1] "Fold 8/10 done"
## [1] "Fold 9/10 done"
## [1] "Fold 10/10 done"
```

and again, use the predict function

```r
predictions = predict(cv_model, test_X)
```

For logistic regression, the `predict` function returns both the predicted class probabilities (`response`) and the predicted class (`class`). We can use this to check the prediction accuracy, given as 82%.

```r
predictions$response[1:5, cv_model$best_lambda_id]
```

```
## [1] 0.4090457 0.6485261 0.1668541 0.1466649 0.3901439
```

```r
predictions$class[1:5, cv_model$best_lambda_id]
```

```
## [1] 0 1 0 0 0
```

```r
sum(predictions$class[, cv_model$best_lambda_id] == test_y)/length(test_y)
```

```
## [1] 0.82
```

Group SLOPE

Group SLOPE (gSLOPE) applies adaptive group penalisation to control the group FDR under orthogonal designs. gSLOPE is described in detail in Brzyski, D., Gossmann, A., Su, W., Bogdan, M. (2019). *Group SLOPE – Adaptive Selection of Groups of Predictors*. gSLOPE is implemented in the `sgs` package with the same features as SGS. Here, we briefly demonstrate how to fit a gSLOPE model.

```r
groups = c(rep(1:20, each = 3), rep(21:40, each = 4), rep(41:60,
    each = 5), rep(61:80, each = 6), rep(81:100, each = 7))
```

```r
data = gen_toy_data(p = 500, n = 400, groups = groups, seed_id = 3)
```

```r
model = fit_gslope(X = data$X, y = data$y, groups = groups, type = "linear",
    lambda = 0.5, gFDR = 0.1, standardise = "l2", intercept = TRUE,
    verbose = FALSE, screen = TRUE)
```

Screening

Screening rules allow the input dimensionality to be reduced before fitting. The strong screening rules for gSLOPE and SGS are described in detail in Feser, F., Evangelou, M. (2024). *Strong Screening Rules for Group-based SLOPE Models*. Here, we demonstrate the effectiveness of screening rules by looking at the speed improvement they provide. For SGS:

```r
screen_time = system.time(model_screen <- fit_sgs(X = data$X,
    y = data$y, groups = groups, type = "linear", path_length = 50,
    alpha = 0.95, vFDR = 0.1, gFDR = 0.1, standardise = "l2",
    intercept = TRUE, verbose = FALSE, screen = TRUE))
no_screen_time = system.time(model_no_screen <- fit_sgs(X = data$X,
    y = data$y, groups = groups, type = "linear", path_length = 50,
    alpha = 0.95, vFDR = 0.1, gFDR = 0.1, standardise = "l2",
    intercept = TRUE, verbose = FALSE, screen = FALSE))
screen_time
```

```
##    user  system elapsed
##    8.85    0.07    9.47
```

```r
no_screen_time
```

```
##    user  system elapsed
##    9.21    0.12    9.59
```

and for gSLOPE:

```r
screen_time = system.time(model_screen <- fit_gslope(X = data$X,
    y = data$y, groups = groups, type = "linear", path_length = 50,
    gFDR = 0.1, standardise = "l2", intercept = TRUE, verbose = FALSE,
    screen = TRUE))
no_screen_time = system.time(model_no_screen <- fit_gslope(X = data$X,
    y = data$y, groups = groups, type = "linear", path_length = 50,
    gFDR = 0.1, standardise = "l2", intercept = TRUE, verbose = FALSE,
    screen = FALSE))
screen_time
```

```
##    user  system elapsed
##    4.56    0.10    5.01
```

```r
no_screen_time
```

```
##    user  system elapsed
##    6.25    0.06    6.48
```

Reference

- Brzyski, D., Gossmann, A., Su, W., Bogdan, M. (2019). *Group SLOPE – Adaptive Selection of Groups of Predictors*.
- Feser, F., Evangelou, M. (2023). *Sparse-group SLOPE: adaptive bi-level selection with FDR-control*.

- Feser, F., Evangelou, M. (2024). *Strong screening rules for group-based SLOPE models.*